ISA vs Microarchitecture

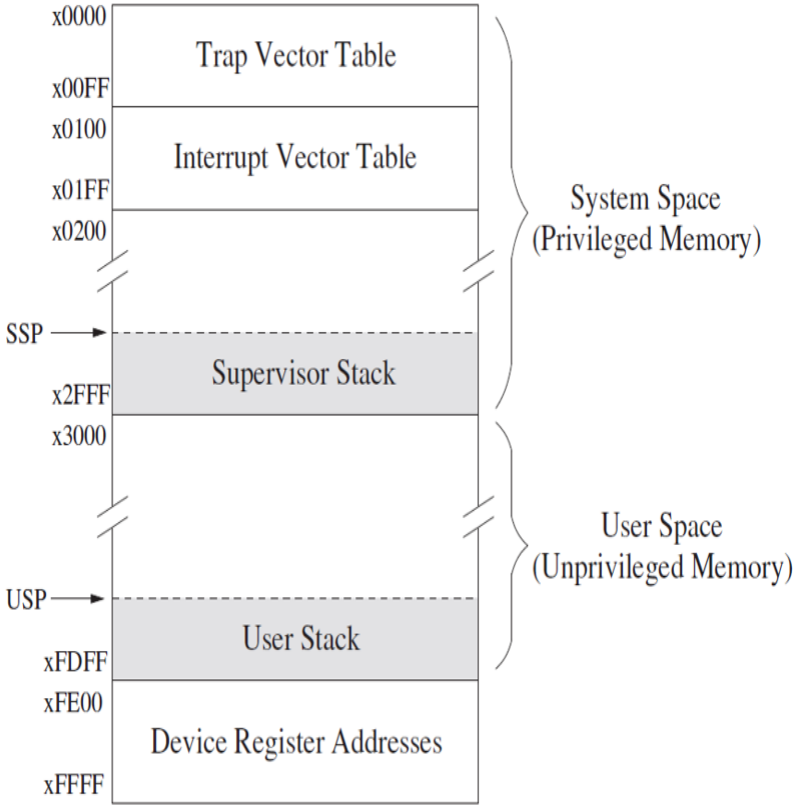
First LC-3 Program: Use of Conditional Branches for Looping

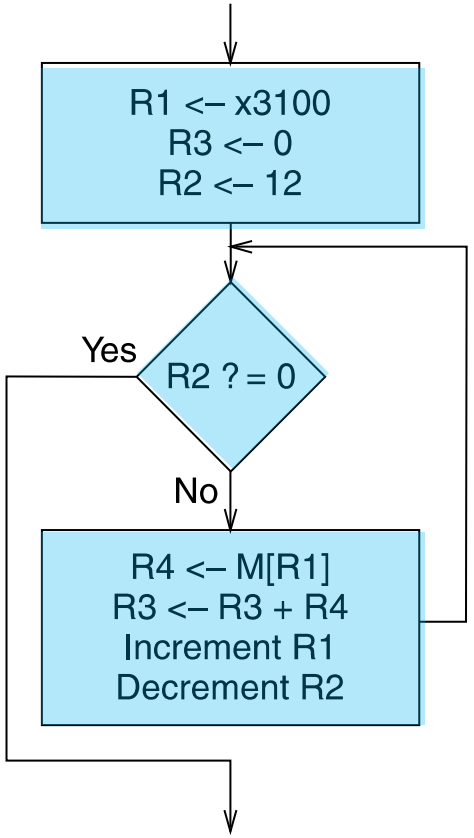
Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel
  - Chapter 5, 6
  - Appendix A – LC3 ISA

An Algorithm for Adding Integers

- We want to write a program that adds 12 integers
  - They are stored in addresses 0x3100 to 0x310B
  - Let us take a look at the flowchart of the algorithm





R1: initial address of integers

R3: final result of addition

R2: number of integers left to be added

Check if R2 becomes 0 (done with all integers?)

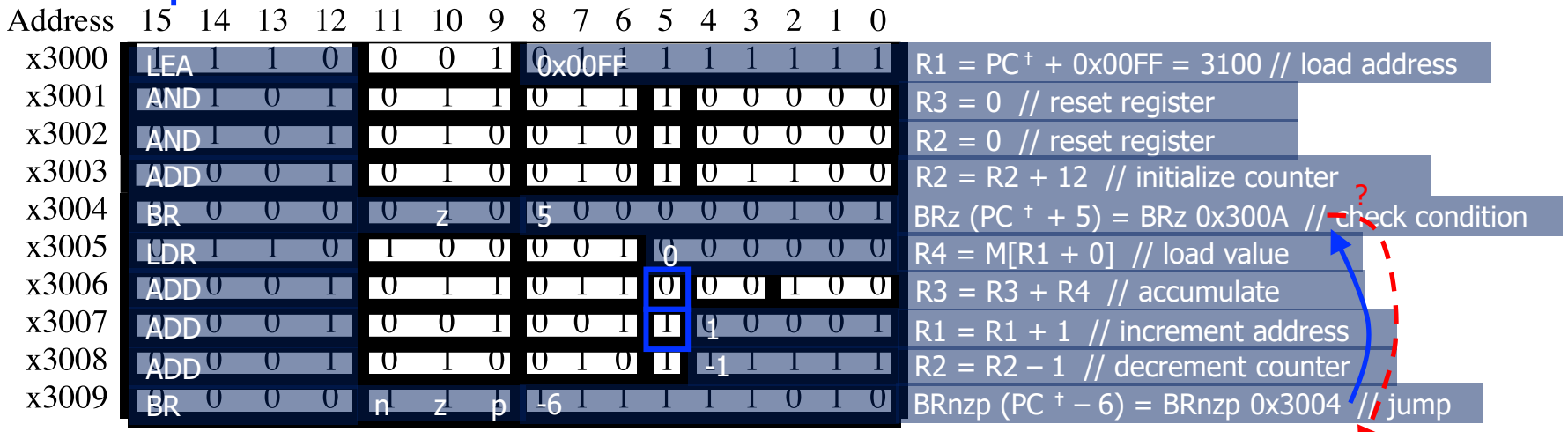
Load integer in R4

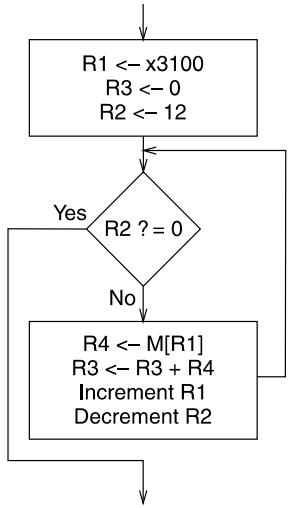
Accumulate integer value in R3

Increment address R1
Decrement R2

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop



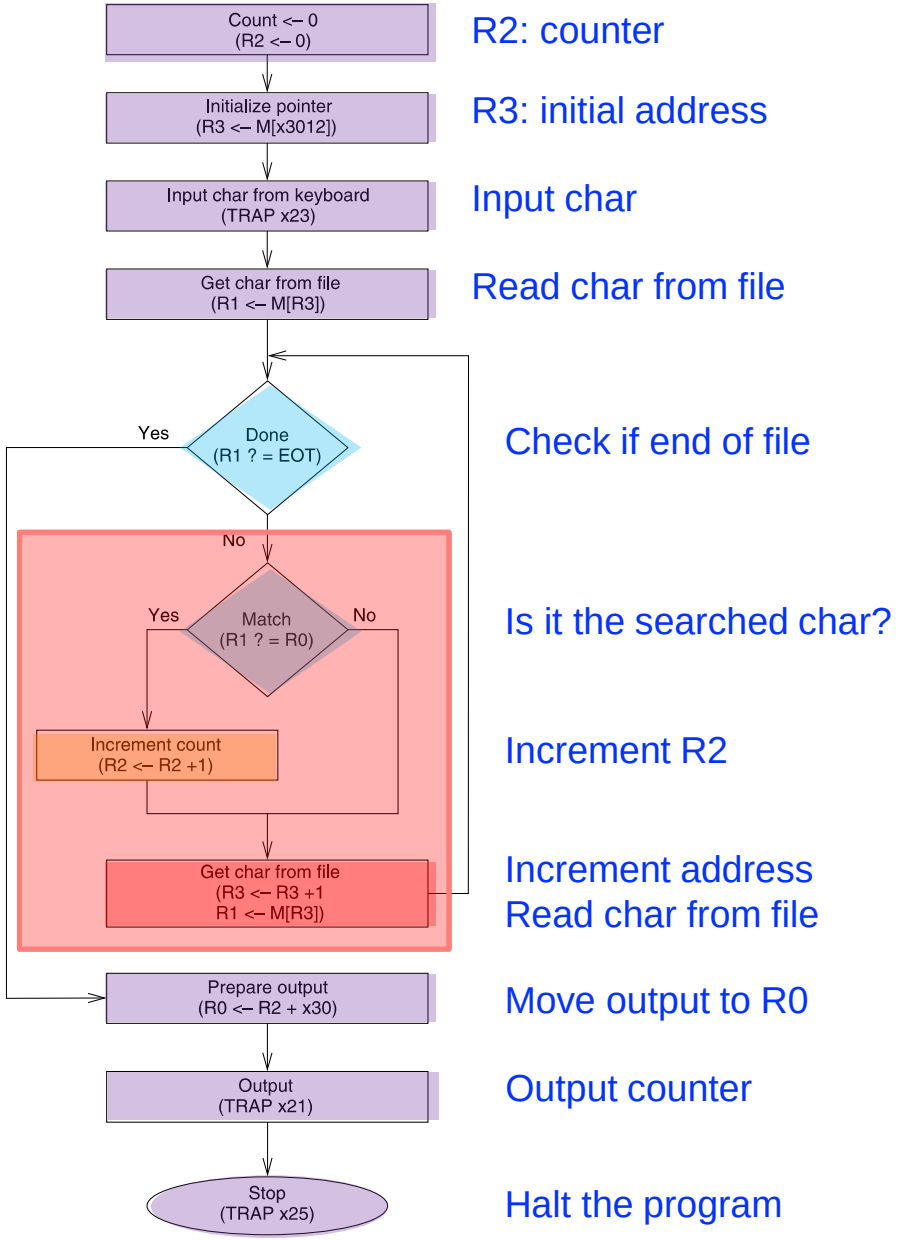


Bit 5 to differentiate both ADD instructions

† This is the incremented PC

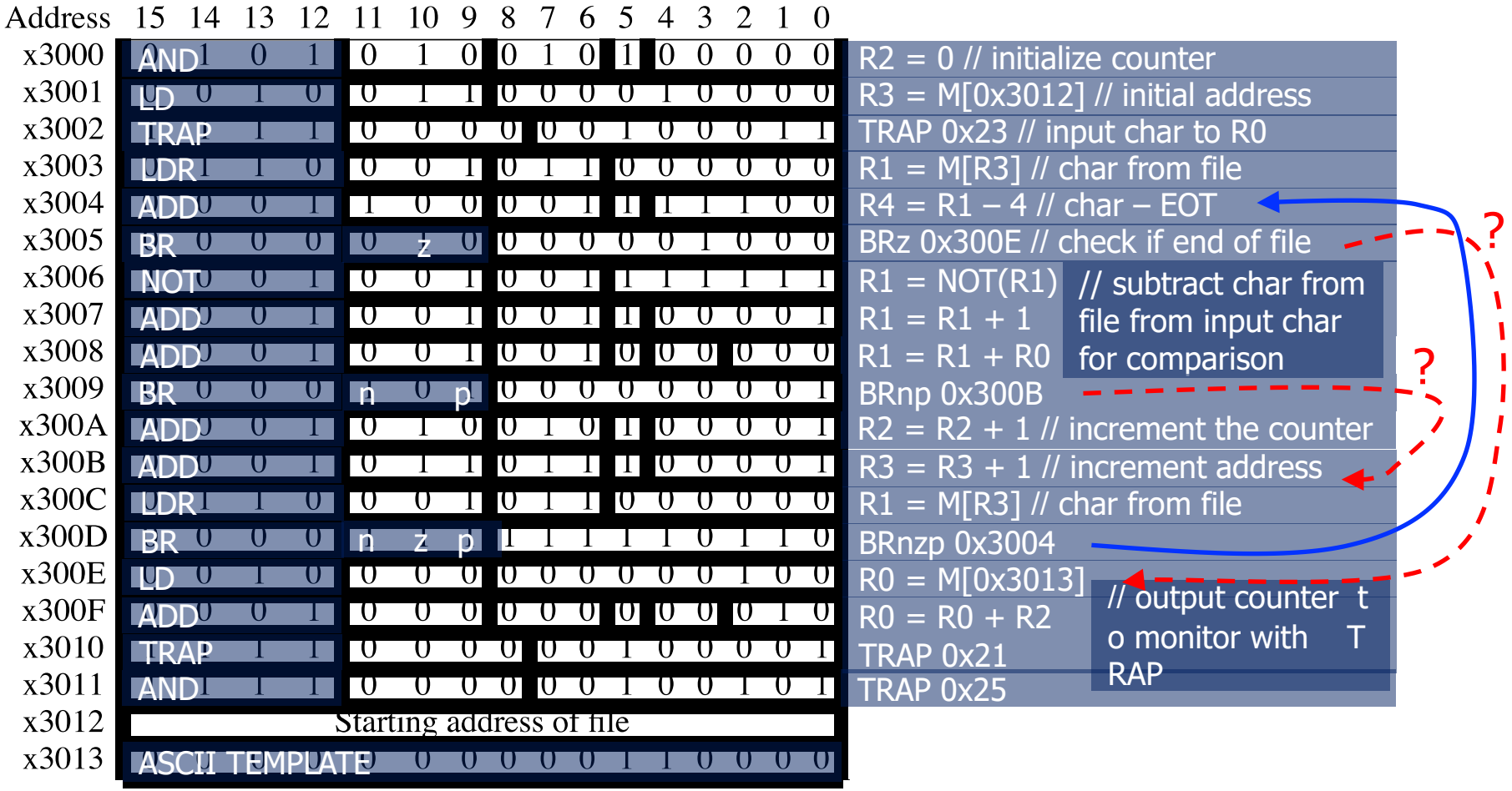
Counting Occurrences of a Character

- We want to write a program that counts the occurrences of a character in a file
  - Character from the keyboard (TRAP instr.)
  - The file finishes with the character EOT (End Of Text)
    - That is called a sentinel
    - In this example, EOT = 4
  - Result to the monitor (TRAP instr.)



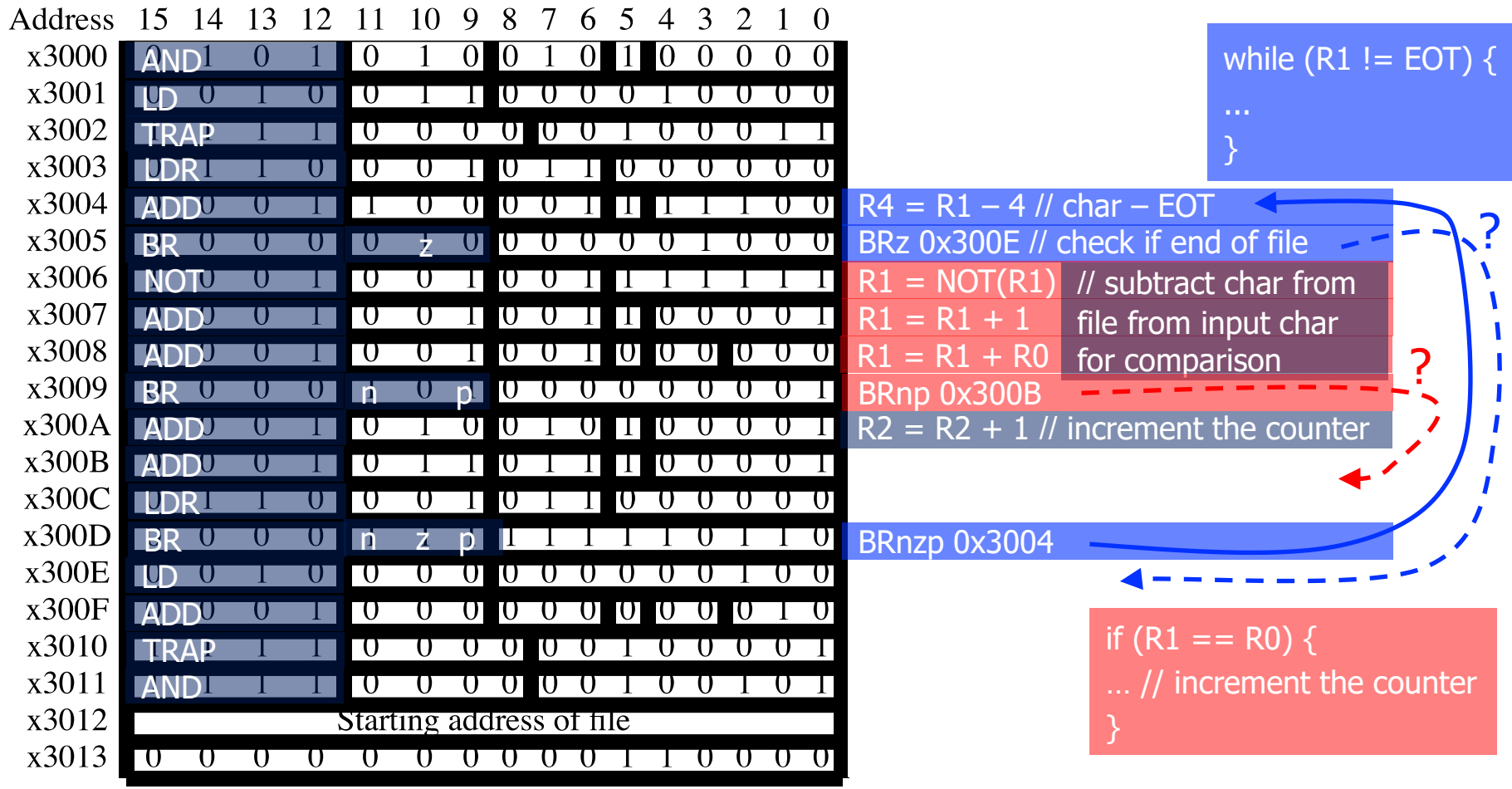
Counting Occurrences of a Char in LC-3

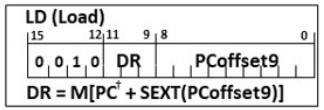
- We use conditional branch instructions to create a loops and if statements

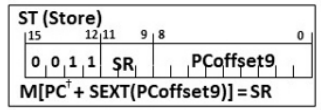


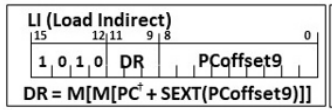
Programming Constructs in LC-3

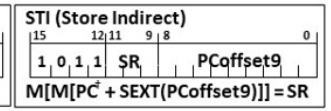
- Let us do some reverse engineering to identify conditional constructs and iterative constructs

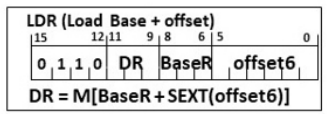


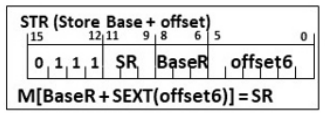


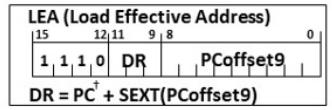


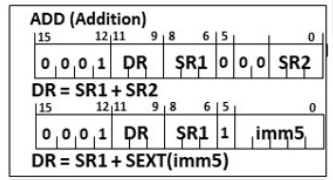


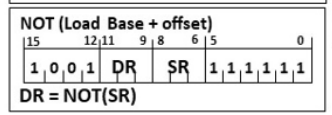


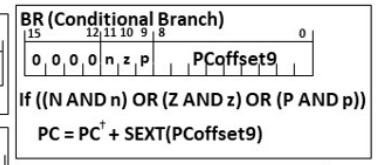


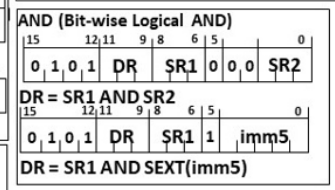


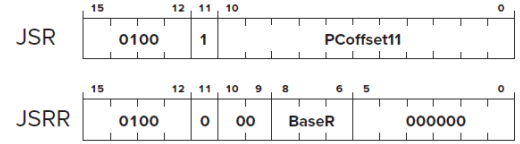


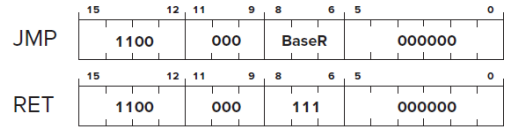


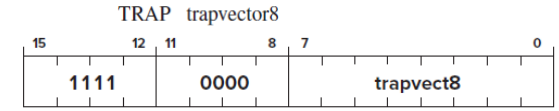












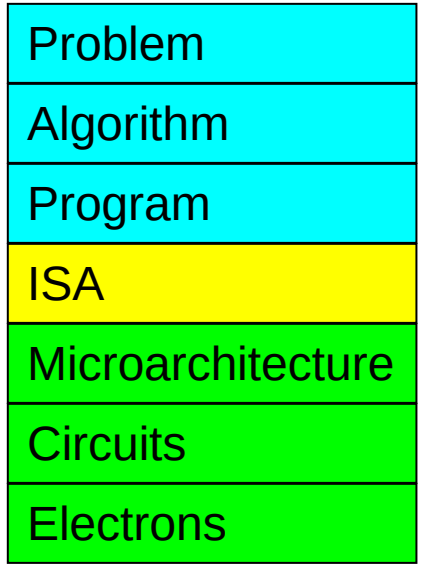
❑trapvect8 = service call

- 0x23 = Input a character from the keyboard
- 0x21 = Output a character to the monitor
- 0x25 = Halt the program

x3000																
x3001																
x3002																
x3003																
x3004																
x3005																
x3006																
X3007																
X3008																
X3009																
X300A																
X300B																
X300C																
X300D																
X300E																
X300F																
X3010																

ISA vs. Microarchitecture

- ISA
  - Agreed upon interface between software and hardware
    - SW/compiler assumes, HW promises
  - What the software writer needs to know to write and debug system/user programs
- Microarchitecture
  - Specific implementation of an ISA
  - Not visible to the software
- "Computer Architecture" = ISA + microarchitecture



ISA vs. Microarchitecture

- What is part of ISA vs. Uarch?
  - Gas pedal: interface for “acceleration”
  - Internals of the engine: implement “acceleration”
- Implementation (uarch) can be various as long as it satisfies the specification (ISA)
  - Add instruction vs. Adder implementation
    - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture
  - x86 ISA has many implementations: 286, 386, 486, Pentium, Pentium Pro, Pentium 4, Core, …
- Microarchitecture usually changes faster than ISA
  - Few ISAs (x86, ARM, SPARC, MIPS, Alpha) but many uarchs
  - *Why?*

ISA

- Instructions
  - Opcodes, Addressing Modes, Data Types
  - Instruction Types and Formats
  - Registers, Condition Codes
- Memory
  - Address space, Addressability, Alignment
  - Virtual memory management
- Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. special instruction
- Task/thread Management
- Multi-threading support, Multiprocessor support

Microarchitecture

- Implementation of the ISA under specific design constraints and goals
- Anything done in hardware without exposure to software
  - Pipelining
  - In-order versus out-of-order instruction execution
  - Memory access scheduling policy
  - Speculative execution
  - Superscalar processing
  - Clock gating
  - Caching - Levels, size, associativity, replacement policy
  - Prefetching

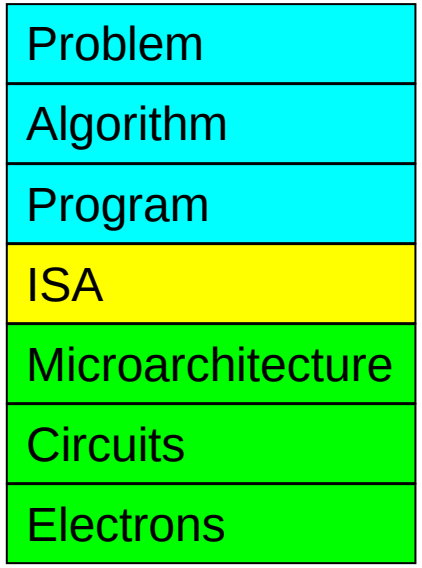
Property of ISA vs. Uarch?

- ADD instruction's opcode?
- Number of general purpose registers?
- Number of ports to the register file?
- Number of cycles to execute the MUL instruction?
- Whether or not the machine employs pipelined instruction execution?

- Remember
  - Microarchitecture: Implementation of the ISA under specific design constraints and goals

Design Point

- A set of design considerations and their importance
  - leads to tradeoffs in both ISA and uarch
- Considerations
  - Cost
  - Performance
  - Maximum power consumption
  - Energy consumption (battery life)
  - Availability
  - Reliability and Correctness
  - Time to Market



- Design point determined by the “Problem” space (application space), the intended users/*market*

Application Space

- Scientific applications
  - Predicting weather
  - Predicting earthquake
- Transaction-based applications
  - ATM transfers
  - E-commerce business
- Network applications
  - High-speed routing of Internet packets
- Real-time applications
  - Meeting critical deadline
- IoT applications
  - Controlling small sensor devices
- Media applications
  - Decoding video and audio files

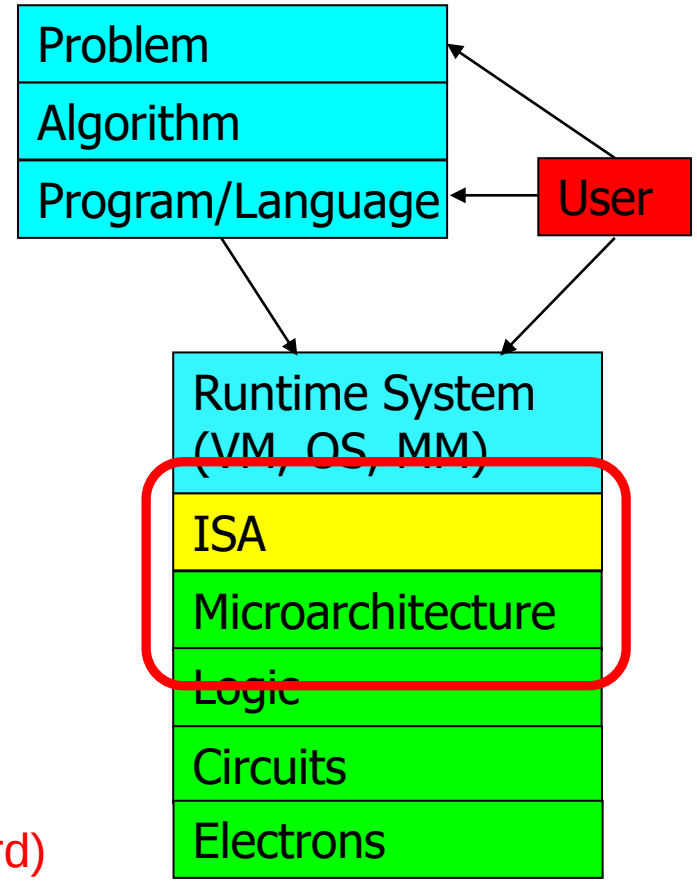
- Cost
- Performance
- Maximum power consumption
- Energy consumption (battery life)
- Availability
- Reliability and Correctness
- Time to Market

Tradeoffs: Soul of Computer Architecture

- ISA-level tradeoffs
- Microarchitecture-level tradeoffs
- System and Task-level tradeoffs
  - How to divide the labor between hardware and software
- *Computer architecture is the science and art of making the appropriate trade-offs to meet a design point*
  - *Why art?*

Why Is It (Somewhat) Art?

Changing demands
at the top
(Look Up and Forward)



Changing demands and
personalities of users
(Look Up and Forward)

Changing issues and
capabilities
at the bottom
(Look Down and Forward)

We do not (fully) know the future (applications, users, market)

Many Different ISAs Over Decades

- x86
- PDP-x: Programmed Data Processor (PDP-11)
- VAX
- IBM 360
- CDC 6600
- SIMD ISAs: CRAY-1, Connection Machine
- VLIW ISAs: Multiflow, Cydrome, IA-64 (EPIC)
- RISC ISAs: Alpha, MIPS, SPARC, ARM

- What are the fundamental differences?
  - E.g., how instructions are specified and what they do
  - E.g., how complex are the instructions

What Are the Elements of An ISA?

- Instructions
  - Opcode
  - Operand specifiers (addressing modes)
    - How to obtain the operand?
- Data types
  - Definition: Representation of information for which there are instructions that operate on the representation
  - Integer, floating point, character, binary, decimal, BCD
  - Doubly linked list, queue, string, bit vector, stack

Data Type Tradeoffs

- What is the benefit of having more or high-level data types in the ISA?
- What is the disadvantage?

- Think compiler/programmer vs. microarchitect

- Concept of semantic gap
  - Data types coupled tightly to the semantic level, or complexity of instructions

- Example: Early RISC architectures vs. Intel 432
  - Early RISC: Only integer data type
  - Intel 432: Object data type

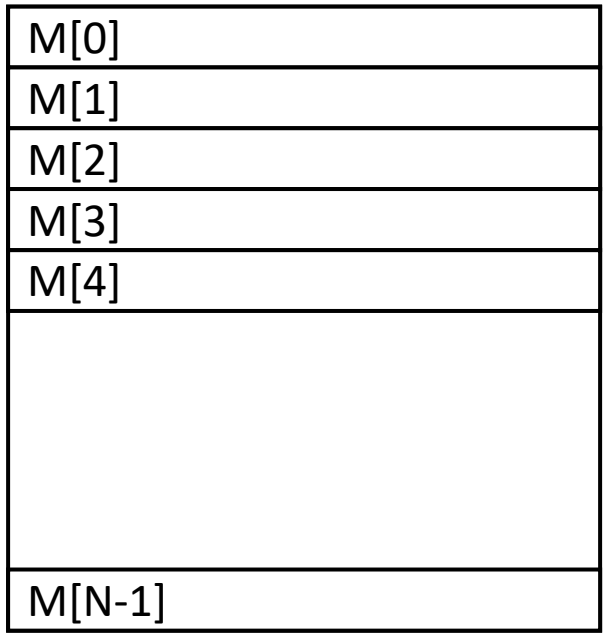
What Are the Elements of An ISA?

- Memory organization
  - Address space: How many uniquely addressable locations in memory
  - Addressability: How much data does each uniquely identifiable location store
    - Byte addressable: most ISAs, characters are 8 bits
    - 32-bit addressable: First Alpha
  - Support for virtual memory

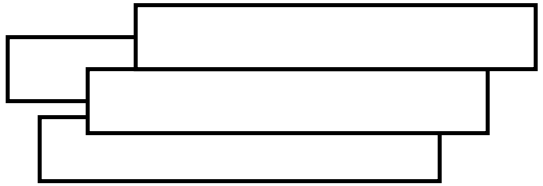
What Are the Elements of An ISA?

- Registers
  - How many
  - Size of each register
- Why is having registers a good idea?
  - Because programs exhibit a characteristic called data locality
  - A recently produced/accessed value is likely to be used more than once (temporal locality)
    - Storing that value in a register eliminates the need to go to memory each time that value is needed

Programmer Visible (Architectural) State



Memory
Array of storage locations indexed by an address



Registers
- given special names in the ISA
  (as opposed to addresses)
- general vs. special purpose

Program Counter
memory address of the current instruction

Instructions specify how to transform the values of programmer visible state
Sequential FSM

Aside: Programmer Invisible State

- Microarchitectural state
- Programmer cannot access this directly

- E.g. cache state

Instruction Classes

- Operate instructions
  - Process data: arithmetic and logical operations
  - Fetch operands, compute result, store result
  - Implicit sequential control flow
- Data movement instructions
  - Move data between memory, registers, I/O devices
  - Implicit sequential control flow
- Control flow instructions
  - Change the sequence of instructions that are executed

What Are the Elements of An ISA?

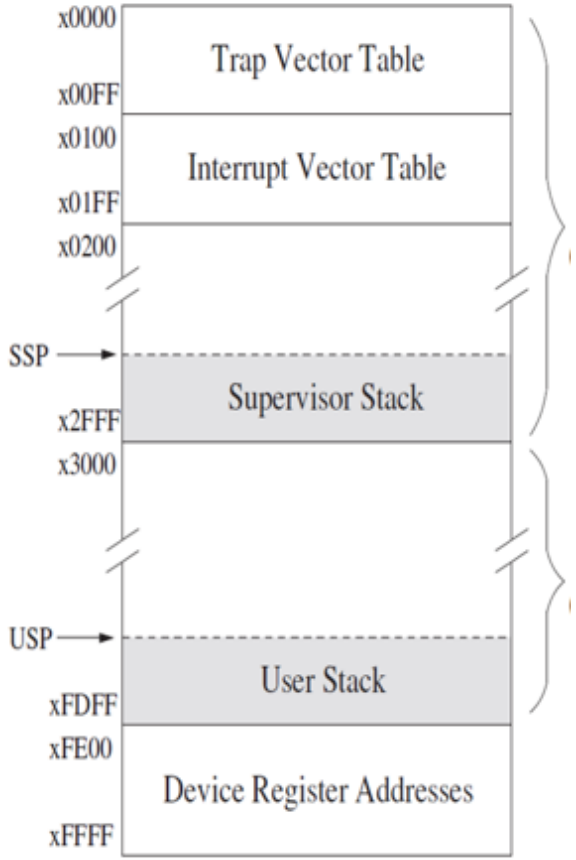
- Load/store vs. memory/memory architectures
  - Load/store architecture: operate instructions operate only on registers
    - E.g., MIPS, ARM and many RISC ISAs
  - Memory/memory architecture: operate instructions can operate on memory locations
    - E.g., x86, VAX and many CISC ISAs

What Are the Benefits of Different Addressing Modes?

- Another example of programmer vs. microarchitect tradeoff
- Advantage of more addressing modes:
  - Enables better mapping of high-level constructs to the machine: some accesses are better expressed with a different mode → reduced number of instructions and code size
    - Think array accesses (autoincrement mode)
    - Think indirection (pointer chasing)
    - Sparse matrix accesses
- Disadvantage:
  - More work for the compiler
  - More work for the microarchitect

What Are the Elements of An ISA?

- How to interface with I/O devices
  - Memory mapped I/O
    - A region of memory is mapped to I/O devices
    - I/O operations are loads and stores to those locations
  - Special I/O instructions
    - IN and OUT instructions in x86 deal with ports of the chip
  - Tradeoffs?
    - Which one is more general purpose?



What Are the Elements of An ISA?

- Privilege modes
  - User vs supervisor
  - Who can execute what instructions?
- Exception and interrupt handling
  - What procedure is followed when something goes wrong with an instruction?
  - What procedure is followed when an external device requests the processor?
  - Vectored vs. non-vectored interrupts (early MIPS)
- Virtual memory
  - Each program has the illusion of the entire memory space, which is greater than physical memory
- Access protection
- We will talk about these later

Complex vs. Simple Instructions

- Complex instruction: An instruction does a lot of work, e.g. many operations
  - Insert in a doubly linked list
  - INDEX instruction (array access with bounds checking)
  - String copy

- Simple instruction: An instruction does small amount of work, it is a primitive using which complex operations can be built
  - Add
  - XOR
  - Multiply

Complex vs. Simple Instructions

- Advantages of Complex instructions
  + Denser encoding → smaller code size → better memory utilization, better cache hit rate (better packing of instructions)
  + Simpler compiler: no need to optimize small instructions as much

- Disadvantages of Complex Instructions
  - Larger chunks of work → compiler has less opportunity to optimize (limited in fine-grained optimizations it can do)
  - More complex hardware → translation from a high level to control signals and optimization needs to be done by hardware

ISA-level Tradeoffs: Semantic Gap

- Where to place the ISA? Semantic gap
  - Closer to high-level language (HLL) → Small semantic gap, complex instructions
  - Closer to hardware control signals? → Large semantic gap, simple instructions

- RISC vs. CISC machines
  - RISC: Reduced instruction set computer
  - CISC: Complex instruction set computer
    - QUICKSORT, FP instructions?
    - VAX INDEX instruction (array access with bounds checking)

ISA-level Tradeoffs: Semantic Gap

- Some tradeoffs

- Simple compiler, complex hardware vs. complex compiler, simple hardware
  - Caveat: Translation (indirection) can change the tradeoff!

- Burden of backward compatibility

- Performance?
  - Optimization opportunity: Example of VAX INDEX instruction: who (compiler vs. hardware) puts more effort into optimization?
  - Instruction size, code size

Small versus Large Semantic Gap

- CISC vs. RISC
  - Complex instruction set computer → complex instructions
    - Initially motivated by “not good enough” code generation
  - Reduced instruction set computer → simple instructions
    - John Cocke, mid 1970s, IBM 801
      - Goal: enable better compiler control and optimization
- RISC motivated by
  - Simplifying the hardware → lower cost, higher frequency
  - Enabling the compiler to optimize the code better
    - Find fine-grained parallelism

How High or Low Can You Go?

- Very large semantic gap
  - Each instruction specifies the complete set of control signals in the machine
  - Compiler generates control signals
  - Open microcode (John Cocke, circa 1970s)
    - Gave way to optimizing compilers
- Very small semantic gap
  - ISA is (almost) the same as high-level language
  - Java machines, LISP machines, object-oriented machines

A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
  - Limited compiler optimization technology
  - Limited memory bandwidth
  - Need for specialization in important applications
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA

Effect of Translation

- One can translate from one ISA to another *ISA* to change the semantic gap tradeoffs
  - ISA (virtual ISA) → Implementation ISA

- Examples
  - Intel's and AMD's x86 implementations translate x86 instructions into programmer-invisible microoperations (simple instructions) in hardware
  - Transmeta's x86 implementations translated x86 instructions into "secret" VLIW instructions in software (code morphing software)

Hardware-Based Translation

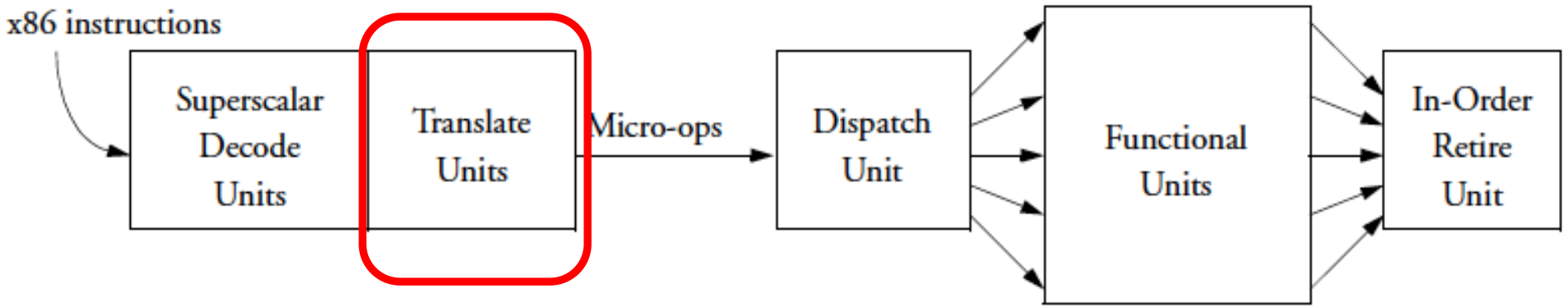


Figure 2. Conventional superscalar out-of-order CPUs use hardware to create and dispatch micro-ops that can execute in parallel.

Klaiber, "The Technology Behind Crusoe Processors," Transmeta White Paper 2000.

Software-Based Translation

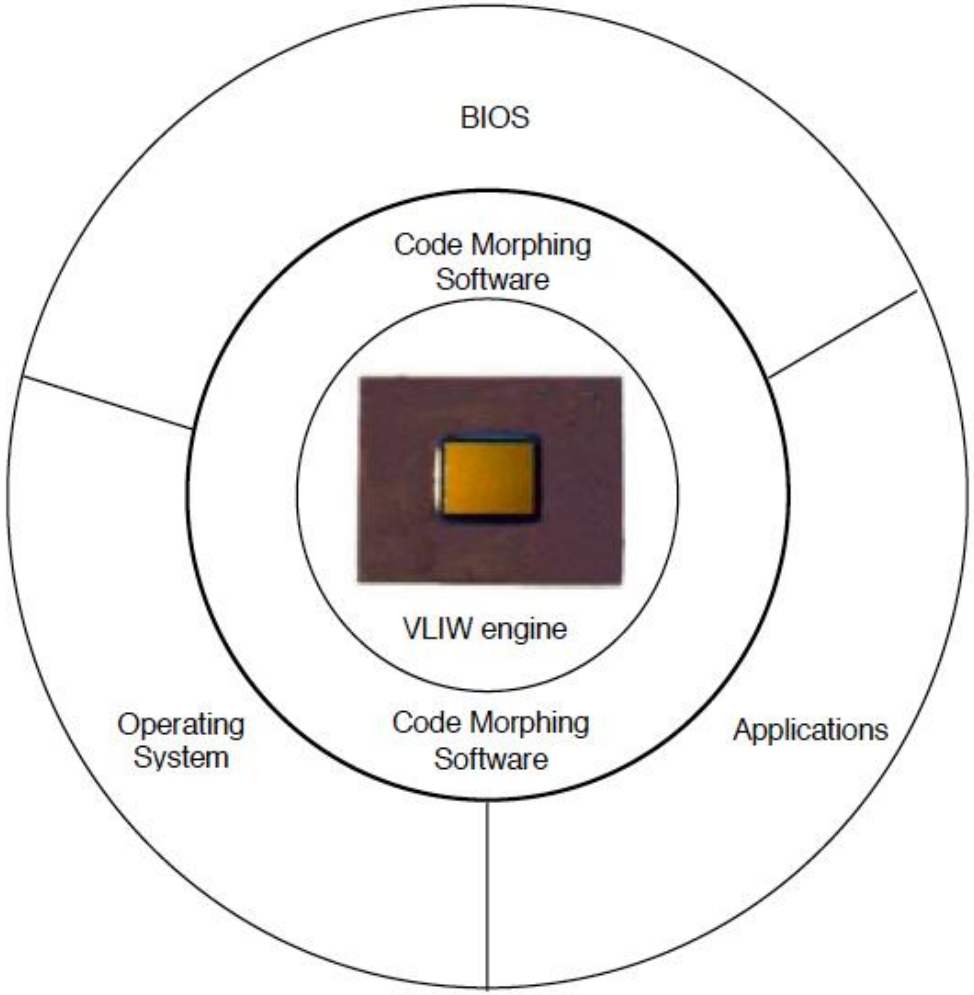


Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Klaiber, "The Technology Behind Crusoe Processors," Transmeta White Paper 2000.

ISA-level Tradeoffs: Instruction Length

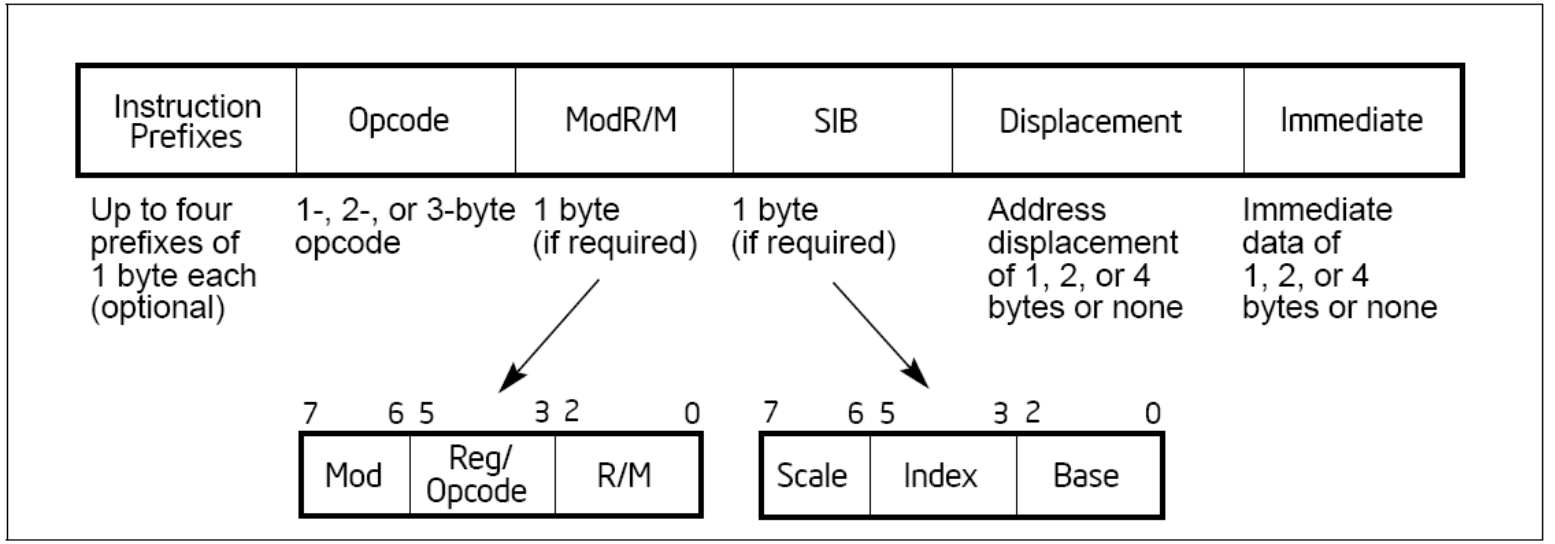
- Fixed length: Length of all instructions the same
  - + Easier to decode single instruction in hardware
  - + Easier to decode multiple instructions concurrently
  - -- Wasted bits in instructions
  - -- Harder-to-extend ISA (how to add new instructions?)
- Variable length: Length of instructions different (determined by opcode and sub-opcode)
  - + Compact encoding
    Intel 432: Huffman encoding (sort of). 6 to 321 bit instructions. How?
  - -- More logic to decode a single instruction
  - -- Harder to decode multiple instructions concurrently
- Tradeoffs
  - Code size (memory space, bandwidth) vs. hardware complexity
  - ISA extensibility and expressiveness vs. hardware complexity
  - Energy? Smaller code vs. ease of decode

ISA-level Tradeoffs: Uniform Decode

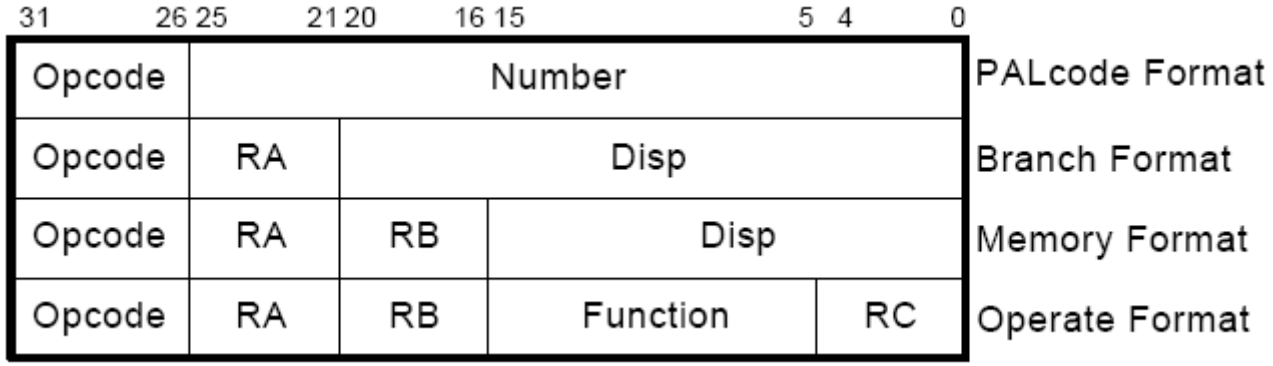
- Uniform decode: Same bits in each instruction correspond to the same meaning
  - Opcode is always in the same location
  - Ditto operand specifiers, immediate values, …
  - Many "RISC" ISAs: Alpha, MIPS, SPARC
  + Easier decode, simpler hardware
  -- Wastes space
- Non-uniform decode
  - E.g., opcode can be the 1st-7th byte in x86
  + More compact and powerful instruction format
  -- More complex decode logic

x86 vs. Alpha Instruction Formats

- x86:

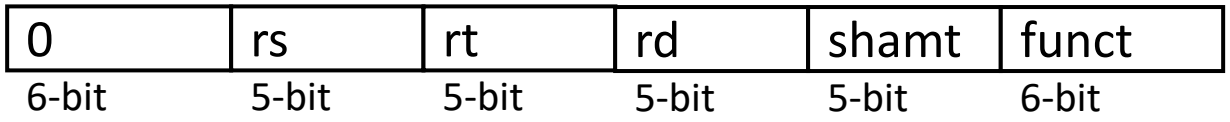


- Alpha:



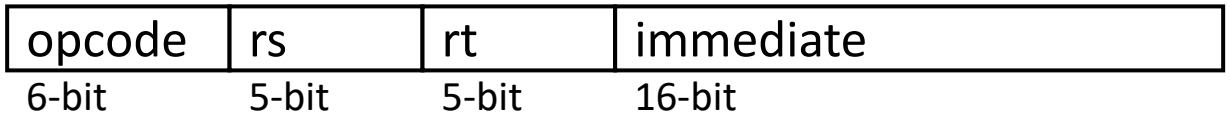
MIPS Instruction Format

- R-type, 3 register operands



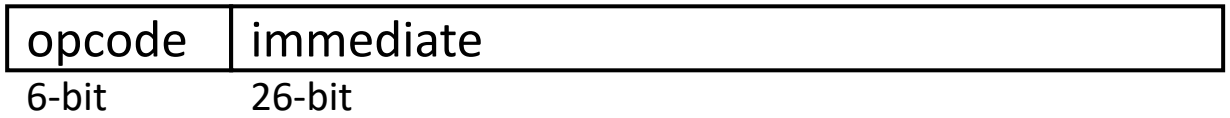
R-type

- I-type, 2 register operands and 16-bit immediate operand



I-type

- J-type, 26-bit immediate operand



J-type

- Simple Decoding
  - 4 bytes per instruction, regardless of format
  - must be 4-byte aligned (2 lsb of PC must be '00')
  - format and fields easy to extract in hardware

A Note on RISC vs. CISC

- Usually, …

- RISC
  - Simple instructions
  - Fixed length
  - Uniform decode
  - Few addressing modes

- CISC
  - Complex instructions
  - Variable length
  - Non-uniform decode
  - Many addressing modes

ISA-level Tradeoffs: Number of Registers

- Affects:
  - Number of bits used for encoding register address
  - Number of values kept in fast storage (register file)
  - (uarch) Size, access time, power consumption of register file

- Large number of registers:
  + Enables better register allocation (and optimizations) by compiler → fewer saves/restores
  -- Larger instruction size

ISA-level Tradeoffs: Addressing Modes

- Addressing mode specifies how to obtain an operand of an instruction
  - Register
  - Immediate
  - Memory (displacement, register indirect, indexed, absolute, memory indirect, autoincrement, autodecrement, …)

- More modes:
  + help better support programming constructs (arrays, pointer-based accesses)
  -- make it harder for the architect to design
  -- too many choices for the compiler?
    - Many ways to do the same thing complicates compiler design

Back to Programmer vs. (Micro)architect

- Many ISA features designed to aid programmers
- But, complicate the hardware designer's job

- Virtual memory
  - vs. overlay programming
  - Should the programmer be concerned about the size of code blocks fitting physical memory?
- Addressing modes
- Unaligned memory access
  - Compiler/programmer needs to align data